

# Lab 9: XiangShan Micro-Architecture

ID : 12532588 NAME : Zihao Li

## 1. 实验说明

本次实验采用非 AI 辅助方式完成：不使用 UCAgent、OpenCode 来生成测试、执行验证、分析结果或撰写结论。仍然使用 Verilator、Picker、Python、shell 这类非 AI 工具完成导出和仿真

## 2. 准备工作

时间开销：32分钟

我先花了32分钟来整理和学习这次 lab 中遇到的概念，例如什么是RTL、Verilog和DUT等等，这样有助于我更好的展开实验。为此，我找了14个参考页面，具体的 url 可以见报告末尾的 list

- Verilator: RTL 仿真基础工具
- Picker: 把 RTL 导出成 Python 可调用工程
- UCAgent: 原题中的 AI 验证 agent，本次不用
- OpenCode: 原题中的 AI 客户端，本次不用
- Adder bug: sum 输出位宽少 1 位

## 3. 执行记录

### 3.1 Basic Command Check

时间开销：1分钟

确认题面会用到的核心命令存在

- git version: git version 2.43.0
- pip3 version: pip 24.0 (python 3.12)
- python3 version: Python 3.12.3
- make version: GNU Make 4.3
- g++ version: g++ 13.3.0

### 3.2 Lab Tool Check

时间开销：18分钟

安装题面要求的 Verilator >= 5.020 以及 Picker 和其依赖，跳过 OpenCode 和 UCAgent 的安装，中间遇到了一些依赖一并安装了

- Verilator version: Verilator 5.020 2024-01-01 rev v5.020
- Picker version: 0.9.0-master-bece7af-2026-04-23

安装命令：

```
sudo apt-get install -y git help2man perl python3 make g++ libfl2 libfl-dev zlib1g zlib1g-dev autoconf flex bison cmake build-essential swig

mkdir -p /home/lzh/CSE5030/tools/src
git clone https://github.com/verilator/verilator.git /home/lzh/CSE5030/tools/src/verilator
cd /home/lzh/CSE5030/tools/src/verilator
git checkout v5.020
autoconf
./configure --prefix=/home/lzh/CSE5030/tools/verilator
make -j$(nproc)
make install

git clone https://github.com/XS-MLVP/picker.git /home/lzh/CSE5030/tools/src/picker
cd /home/lzh/CSE5030/tools/src/picker
make init
PATH=/home/lzh/CSE5030/tools/verilator/bin:$PATH make install ARGS="-DCMAKE_INSTALL_PREFIX=/home/lzh/CSE5030/tools/picker"

export PATH=/home/lzh/CSE5030/tools/picker/bin:/home/lzh/CSE5030/tools/verilator/bin:$PATH
verilator --version
picker --version
```

参考资料：

- Verilator official install guide: <https://verilator.org/guide/latest/install.html>
- Verilator GitHub repository: <https://github.com/verilator/verilator>
- Picker GitHub repository: <https://github.com/XS-MLVP/picker>
- Picker README install instructions: <https://github.com/XS-MLVP/picker#install-from-source>

### 3.3 Picker Export

时间开销：6分钟

将 `Adder/Adder.v` 导出为后续可使用的验证工程。

执行命令：

```
export PATH=/home/lzh/CSE5030/tools/picker/bin:/home/lzh/CSE5030/tools/verilator/bin:$PATH
picker export Adder/Adder.v --rw 1 --sname Adder --tdir output/ -c -w Adder.vcd
ls -l output/Adder
cp Adder/README.md output/Adder/README.md
ls -l output/Adder/README.md
```

执行中先按题面写法尝试了 `-w`，当前 Picker 版本报错：

```
--wave_file_name: 1 required TEXT missing
```

原因是本地 `picker export --help` 显示 `-w, --wave_file_name TEXT` 需要提供波形文件名，所以改为 `-w Adder.vcd` 后重新执行成功

### 3.4 Manual Test Design

时间开销：8分钟

手工设计测试用例，不使用 AI agent 自动生成测试

规格和接口检查：

- `Adder/README.md` 说明输入为 `a`、`b`、`cin`，输出为 `sum`、`cout`
- `Adder/README.md` 说明加法关系为 `sum = a + b + cin`
- `Adder/Adder.v` 中 `a` 和 `b` 是 `[WIDTH-1:0]`，当 `WIDTH = 64` 时为 64 位
- `Adder/Adder.v` 中 `sum` 是 `[WIDTH-2:0]`，当 `WIDTH = 64` 时只有 63 位
- `output/Adder/signals.json` 也显示 `sum` 的 `High` 为 62，说明导出后的接口同样只有 63 位

测试点记录：

| Case            | a                  | b                  | cin | 目的                                         |
|-----------------|--------------------|--------------------|-----|--------------------------------------------|
| normal add      | 0x0000000000000001 | 0x0000000000000002 | 0   | 普通不进位加法，检查基础路径                             |
| carry out       | 0xffffffffffffffff | 0x0000000000000000 | 1   | 检查 64 位加法溢出时的 <code>cout</code>            |
| high result bit | 0x7fffffffffffffff | 0x0000000000000001 | 0   | 触发正确 64 位 <code>sum</code> 的最高有效位，即 bit 63 |

expected 计算方式：

```
full = a + b + cin
expected_sum = full & 0xffffffffffffffff
expected_cout = (full >> 64) & 0x1
```

预期结果：

| Case            | expected sum       | expected cout |
|-----------------|--------------------|---------------|
| normal add      | 0x0000000000000003 | 0             |
| carry out       | 0x0000000000000000 | 1             |
| high result bit | 0x8000000000000000 | 0             |

### 3.5 Simulation Run

时间开销：26分钟

运行手写测试脚本，记录真实命令和输出

执行命令：

```
cd /home/lzh/CSE5030/lab9/output/Adder
python3 manual_test.py
```

输出摘要：

```
normal add: a=0x0000000000000001 b=0x0000000000000002 cin=0 expected_sum=0x0000000000000003 actual_sum=0x0000000000000003 expected_cout=0 actual_cout=0
result=PASS
carry out: a=0xffffffffffffffff b=0x0000000000000000 cin=1 expected_sum=0x0000000000000000 actual_sum=0x0000000000000000 expected_cout=1 actual_cout=0
result=FAIL
high result bit: a=0x7fffffffffffffff b=0x0000000000000001 cin=0 expected_sum=0x8000000000000000 actual_sum=0x0000000000000000 expected_cout=0
actual_cout=1 result=FAIL
summary: total=3 failed=2
```

命令退出码为 1，表示手写测试中存在失败 case。

### 3.6 Failed Case Record

时间开销：4分钟

记录两组失败输入

| Case            | a                  | b                  | cin | expected sum       | actual sum         | expected cout | actual cout |
|-----------------|--------------------|--------------------|-----|--------------------|--------------------|---------------|-------------|
| carry out       | 0xffffffffffffffff | 0x0000000000000000 | 1   | 0x0000000000000000 | 0x0000000000000000 | 1             | 0           |
| high result bit | 0x7fffffffffffffff | 0x0000000000000001 | 0   | 0x8000000000000000 | 0x0000000000000000 | 0             | 1           |

carry out 这组 case 的完整结果应为 65 位的 0x1\_0000000000000000。正确设计应输出 sum = 0x0000000000000000、cout = 1，但当前 {cout, sum} 只有 64 位，真正的第 64 位 carry 被截断，所以 actual cout = 0。

high result bit 这组 case 的完整结果是 0x8000000000000000，没有超过 64 位，所以正确 cout 应为 0。但当前 RTL 的 sum 只有 63 位，结果 bit 63 被拼接表达式 {cout, sum} 接到了 cout 上，所以 actual sum = 0x0000000000000000、actual cout = 1。

## 4. Bug 分析

时间开销：12分钟

### 4.1 Error Description

规格中 Adder 应该输出 64 位 sum 和 1 位 cout。但是代码中 sum 被声明成 [WIDTH-2:0]，当 WIDTH = 64 时只有 63 位

由于赋值语句是 {cout, sum} = a + b + cin;，左侧总宽度变成 1 + 63 = 64 位，而正确的完整加法结果需要 65 位。这样会产生两个问题：

1. 正确结果的 bit 63 会进入 cout，而不是进入 sum
2. 真正的 65 位 carry out 会被截断

第一个失败 case 是 0xffffffffffffffff + 0 + 1。正确完整结果是 0x1\_0000000000000000，expected cout = 1，但 actual cout = 0。这说明真正的第 64 位 carry out 被截断了。

第二个失败 case 是 0x7fffffffffffffff + 1。正确结果还没有超过 64 位，expected sum = 0x8000000000000000、expected cout = 0，但 actual sum = 0x0000000000000000、actual cout = 1。这说明正确结果的 bit 63 被错误接到了 cout。

### 4.2 Correct Design Should Be

正确设计应改成：

```
output [WIDTH-1:0] sum,
```

[WIDTH-1:0] 表示从第 WIDTH-1 位到第 0 位，共 WIDTH 位。这样 {cout, sum} 左侧总宽度才是 WIDTH + 1，可以完整接住 a + b + cin 的结果。

## 5. 手工 Workflow Summary

时间开销：5分钟

我先阅读题面并区分原始流程中的 AI 自动化部分和本次可以使用的非 AI 工具。根据老师要求，本次没有安装、配置或使用 UCAgent、OpenCode、MCP 或其他大模型工具。

实验中我手工准备了 `Adder/Adder.v` 和 `Adder/README.md` , 然后使用 `Picker` 和 `Verilator` 将 RTL 导出成 Python 可调用的仿真工程。导出时当前 `Picker` 版本要求 `-w` 后面提供波形文件名, 所以最终使用 `-w Adder.vcd` 完成导出。

之后我手工对照规格和 RTL, 设计了普通加法、进位输出和最高结果位三个测试点, 并手写 `output/Adder/manual_test.py` 运行仿真。仿真结果中普通加法通过, 但 `carry out` 和 `high result bit` 两个 case 失败。最后我根据 `expected / actual` 对比和源码声明确认 bug 来自 `sum` 位宽写成 `[WIDTH-2:0]` , 导致 `{cout, sum}` 左侧总宽度不足。

## 6. 用时分析

本次总用时估计为 112 分钟。主要时间消耗在背景资料整理、工具安装和手写仿真脚本上

| 阶段                  | 用时    |
|---------------------|-------|
| 准备工作与概念学习           | 32 分钟 |
| Basic Command Check | 1 分钟  |
| Lab Tool Check      | 18 分钟 |
| Picker Export       | 6 分钟  |
| Manual Test Design  | 8 分钟  |
| Simulation Run      | 26 分钟 |
| Failed Case Record  | 4 分钟  |
| Bug 分析              | 12 分钟 |
| Workflow Summary    | 5 分钟  |

## 7. Reference list

- UCAgent README: <https://github.com/XS-MLVP/UCAgent/blob/main/README.zh.md>
- UCAgent paper: <https://arxiv.org/pdf/2603.25768>
- Picker GitHub: <https://github.com/XS-MLVP/picker>
- Picker / UnityChip 安装文档: <https://open-verify.cc/mlvp/en/docs/quick-start/installer/>
- Toffee GitHub: <https://github.com/XS-MLVP/mlvp>
- Verilator install: <https://verilator.org/guide/latest/install.html>
- Verilator overview: <https://verilator.org/guide/latest/overview.html>
- MCP intro: <https://modelcontextprotocol.io/docs/getting-started/intro>
- MCP specification: <https://modelcontextprotocol.io/specification/2025-03-26>
- OpenCode config: <https://opencode.ai/docs/config/>